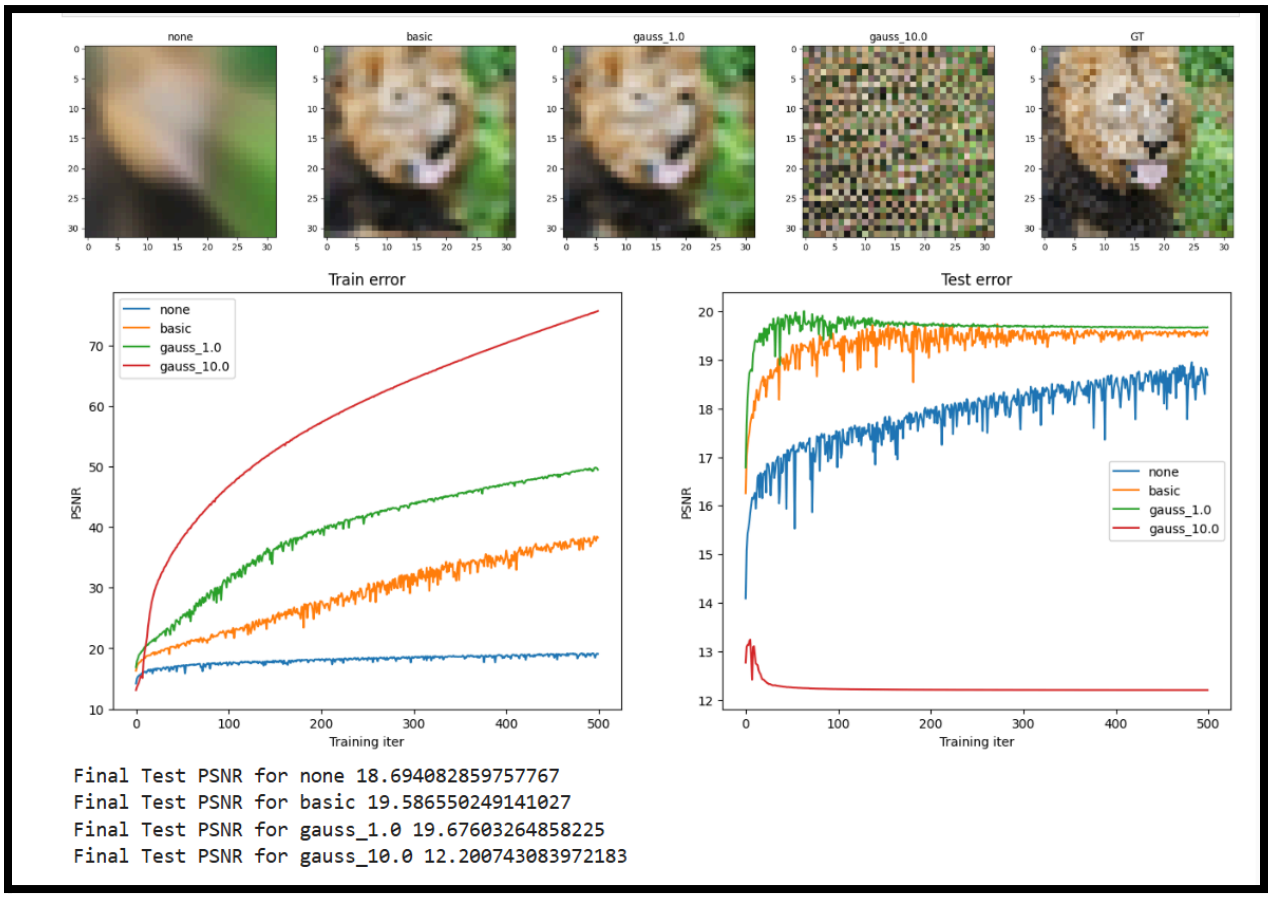
Assignment 2 Report

Name(s): Vinay Krishnan and Ryed Badr
NetID(s): vinayek2 and rbadr2

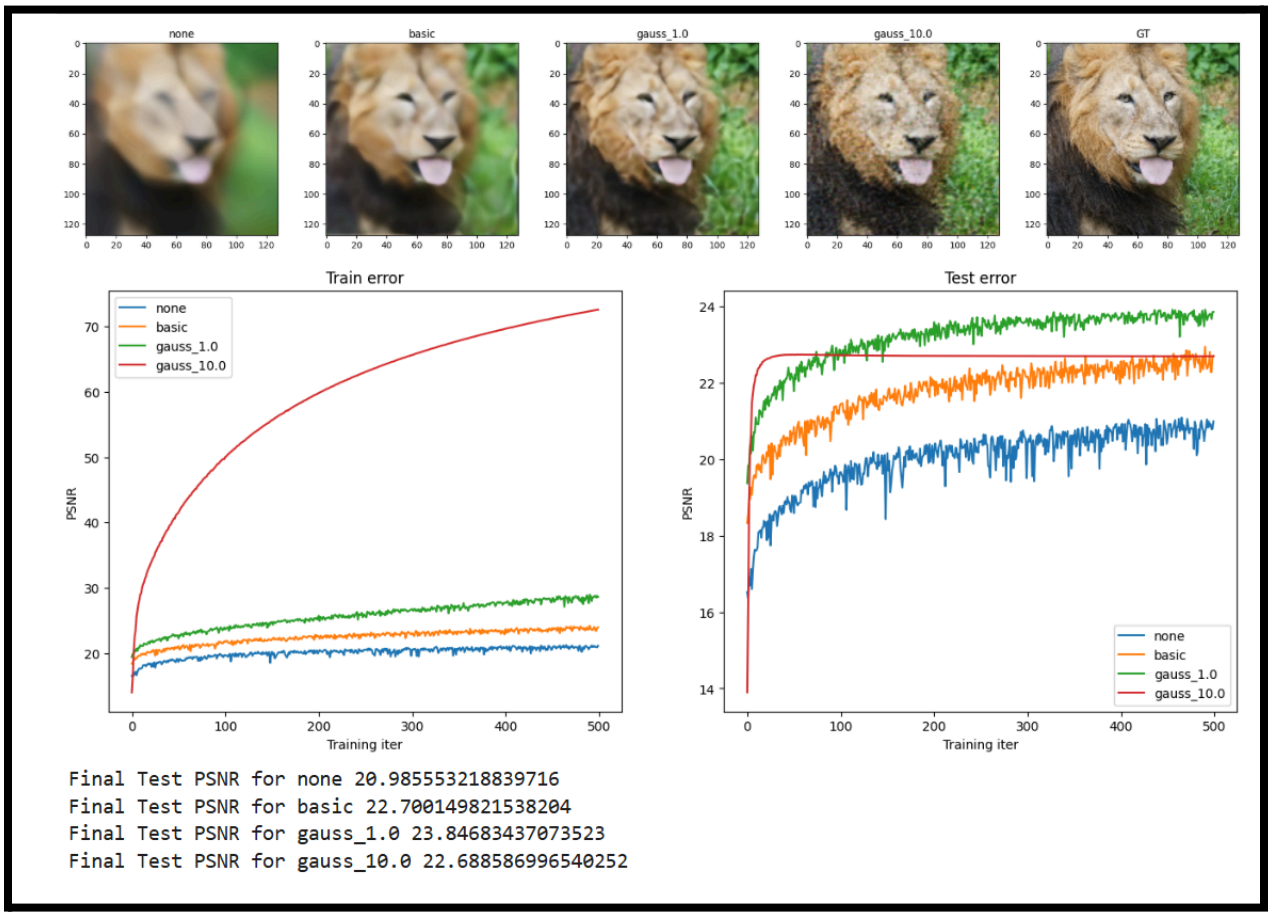
In each the following parts, you should make sure the following are included:

- Train and test PSNR plots
- Image reconstructions
    - For the example image: None mapping, basic mapping and Fourier feature mappings (sigma = 1, 10)
    - For your image: Only the best mapping you found from the prior experiments
- The ground truth image

Part 1: Low resolution example

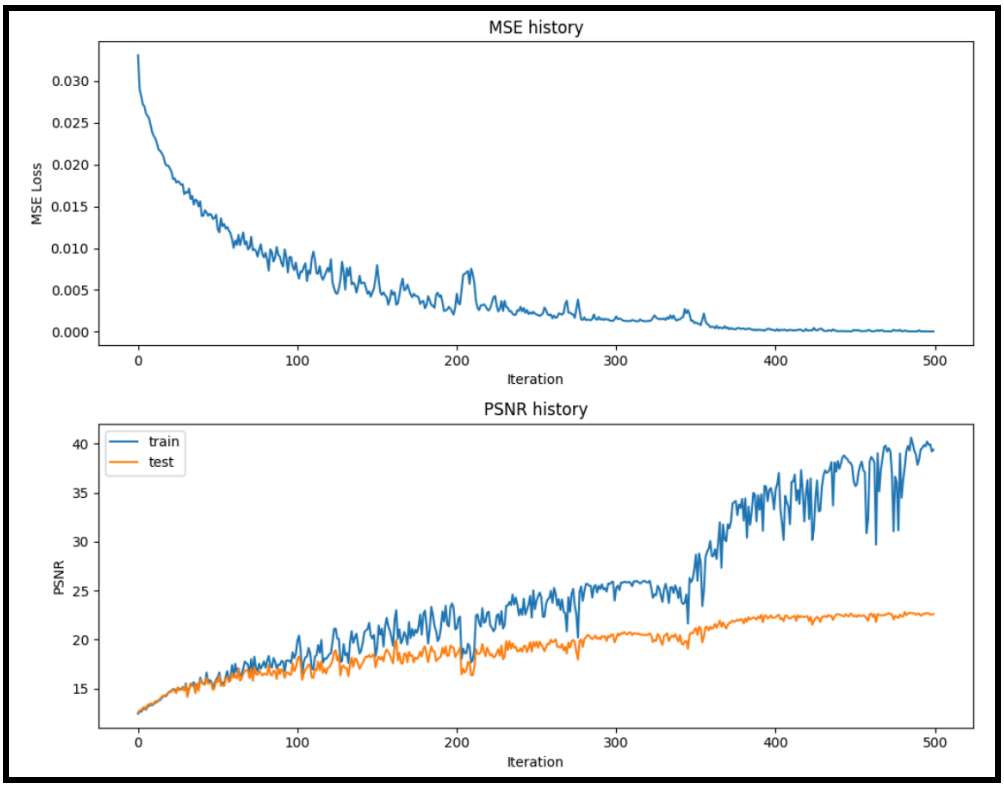


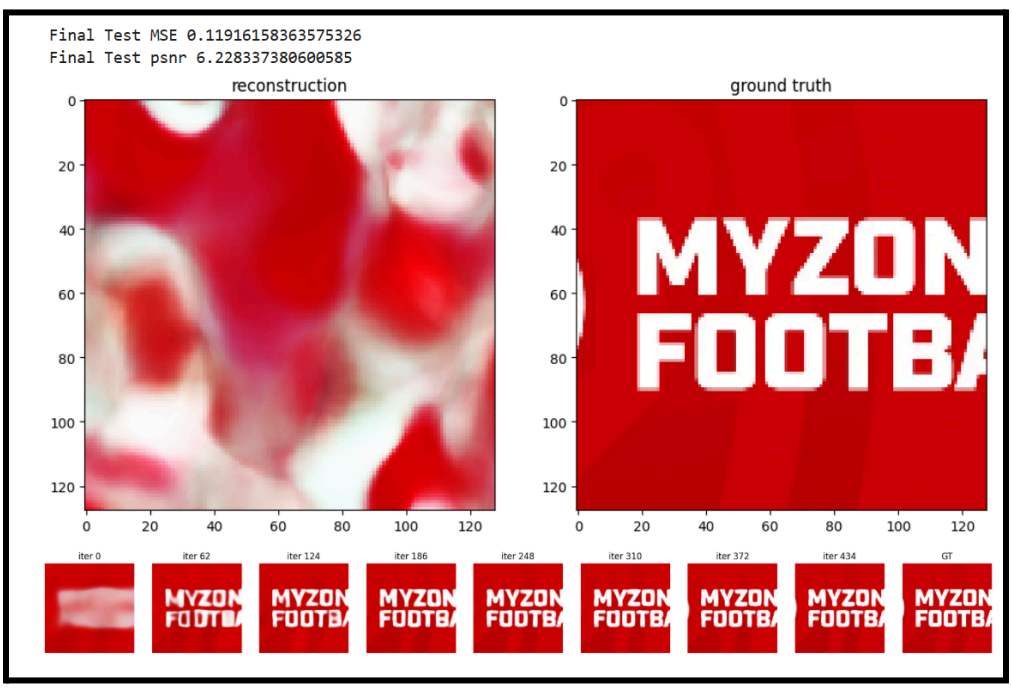
Part 2: High resolution example



Part 3: High resolution (image of your choice)

Gauss 1.0 Mapping





Part 4: Discussion

How did the different choices for coordinate mappings functions compare? Why do you think that the results for sigma = 10 are not as good as for sigma = 1?

The different choices for coordinate mappings include: None, Basic, Gauss 1, and Gauss 10. Each mapping holds various PSNR scores that indicate better image or video reconstruction quality with certain higher values signifying that a reconstructed version of an image closely resembles its original. For None mapping, this specifically represents the baseline measurement which suggests that the network can only see raw coordinates which means that it actually struggles to learn sharp edges resulting in a rather blurry "low-pass" reconstruction. For Basic mapping, we add a single frequency which provides our network to have a rather "periodic" view of the data, aiding its ability to resolve slightly finer details. For Gauss 1.0, this represents the experiment's natural sweet spot as the mapping shows a diverse set of frequencies that match and take into account the natural detail level of the lion image. It actually generalizes quite well to test pixels. Unfortunately, the same cannot be said for Gauss 10. It actually has the potential to see the sharpest of pixels but somehow it performed worse in the baseline. This is most likely due to the fact that we are working with a $\sigma = 10$ which indicates that our B matrix contains large random numbers. This contains extremely high-frequency sine waves that "wiggle" faster than the distance between our training pixels. Meaning that when the network tries to predict a test pixel between training points the frequency of the wave is in a completely different phase which results in "noisy" prediction. This actually demonstrates overfitting within the frequency domain.

What interesting observation(s) did you make regarding the train and test plots?

The training PSNR increases much quicker and to a bigger value than the testing PSNR. That being said, the test curve still follows a similar pattern which means that the model is still generalizing to the unseen points. Further the Gaussian mappings tended to converge faster than the ones with no mapping or the basic mapping, however the Gauss 1.0 mapping achieved much higher PSNR scores than Gauss 10.0. For both training and testing plots, they all increase and converge.

What insight(s) did you gain from your own image example (Part 3)?

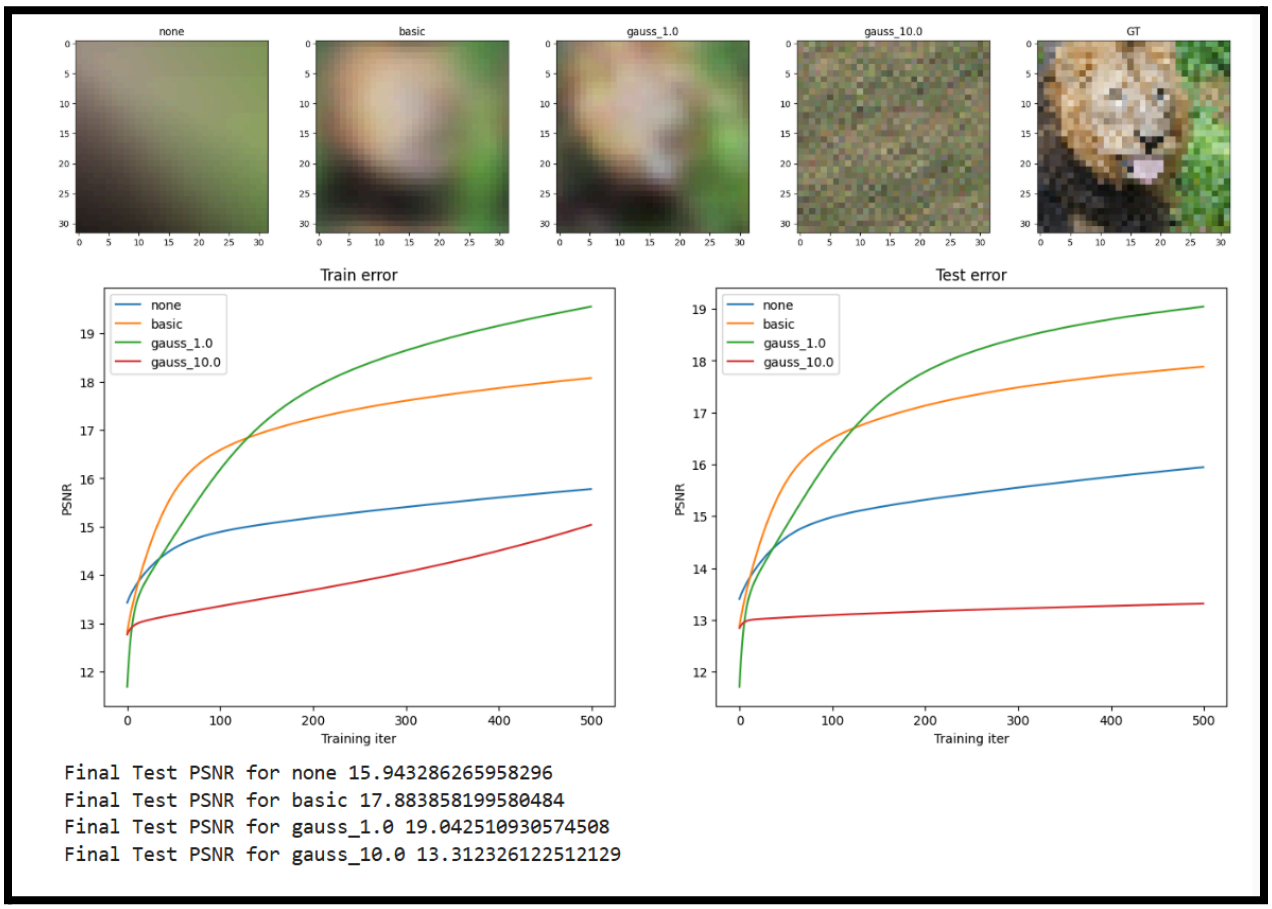
Some insights we gained by looking at the reconstruction of our own image example is that it is difficult for coordinate-based networks to mirror or build images that have high contrast text and sharp edges. Based on our PSNR history graph using Gauss 1 as our mapping, there is actually a massive divergence between the train (blue) and test (orange) lines. This shows that our model is actually achieving a very high PSNR on the training pixels which is around 40 but the test plateaus much lower. This makes sense considering that text itself is considered mathematically "sharp" since we're transitioning from a red color and white instantly based on our image example. This means that the model must perfectly align its sine waves to reconstruct this. The model is most likely hallucinating the transitions since the font has not been learned before in the previous example, only the specific color of the points it was shown. It's also important to know that $\sigma = 1$ is probably too low of a frequency. While it provides enough detail to represent the lion's fur by smoothening the gradients, but not enough to represent the "sharpness" for the crisp edges represented in our training example.

Part 5: Extra Credit (optional)

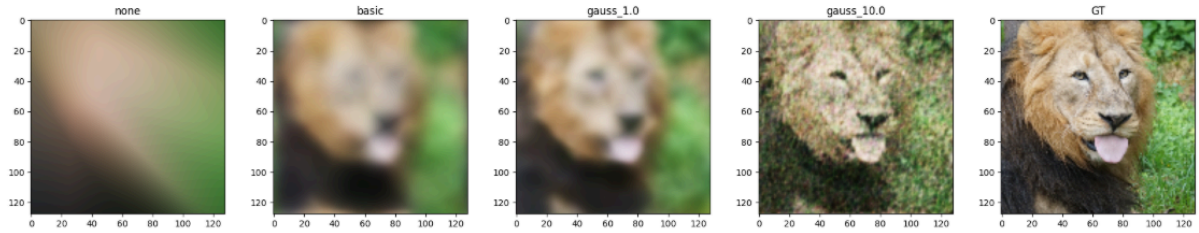
Explain anything you did in detail and provide output images.

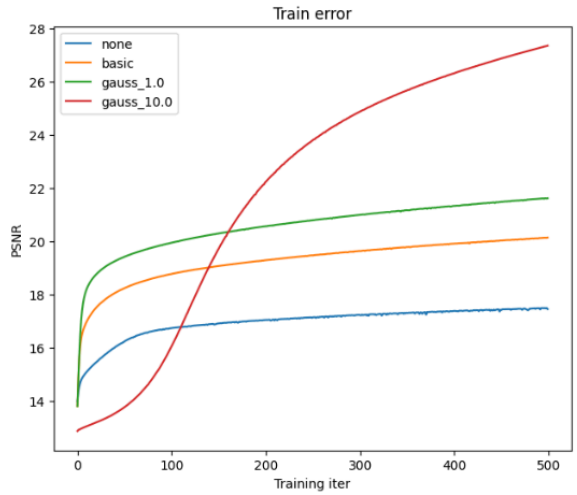
For extra credit, we decided to implement the Adam optimizer. In order to accomplish this, we needed to do research on the necessary equations and how the optimizer actually worked. We found that it stores two running statistics for each parameter, called the first moment (m) and the second moment (v). During each update step, the gradient is used to update these moving averages, which estimate the mean and variance of past gradients. Bias correction is applied using the timestep (t), and parameters are updated using the rule $\theta = \theta - lr * \frac{\widehat{m}}{\sqrt{\hat{v}+\epsilon}}$. This gives us the update rule. To calculate each moment, we used the equation $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ where mt is the first moment estimate, gt is the current gradient and b1 is the decay rate. The second moment calculation was similar except that the gt was squared and b2 was used instead of b1. To get the bias corrected versions, we needed to divide each of those moments by $1 - \beta_i^t$ where i is the number of the moment it belonged to. We implemented this into code, tested, and saw generally lower test PSNR values.

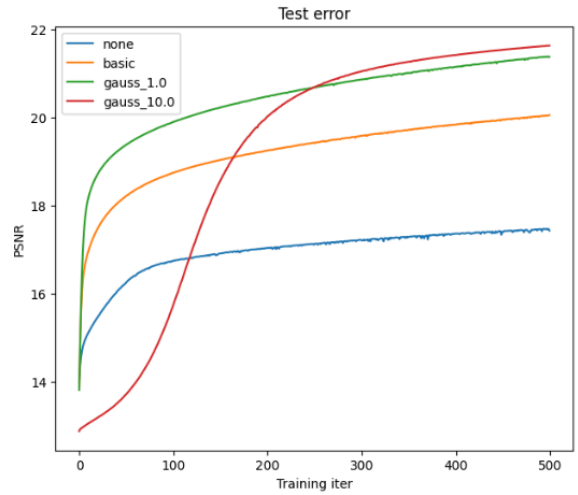
Low Resolution using Adam:



High Resolution using Adam:








```
Final Test PSNR for none 17.43561900952877
Final Test PSNR for basic 20.05566131414344
Final Test PSNR for gauss_1.0 21.37784666640485
Final Test PSNR for gauss_10.0 21.629921530227474
```